

# Resumo para Prova — Programação Funcional (Haskell)

Foco: List Comprehensions, Recursão, Funções de Alta Ordem (map/filter/fold) e Lambdas

---

## 1. List Comprehensions

Sintaxe geral:

```
[ expressão | geradores, condições ]
```

### 1.1 Geradores simples

```
[x^2 | x <- [1..100]]           -- gera 12, 22, ..., 1002
```

**Exercício 1 da lista** — soma  $1^2+2^2+\dots+100^2$ :

```
soma = sum [x^2 | x <- [1..100]]
```

### 1.2 Múltiplos geradores

```
[(x,y) | x <- [1,2,3], y <- [4,5,6]]  
-- = [(1,4), (1,5), (1,6), (2,4), (2,5), (2,6), (3,4), (3,5), (3,6)]
```

A ordem dos geradores importa para a ordem do resultado (o último gerador “varia mais rápido”, como um loop interno).

**Exercício 5** — reescrever com geradores únicos + concat:

```
-- Comprehension aninhada (gera lista de listas, um sub-resultado por x)  
passo1 = [(x,y) | y <- [4,5,6] | x <- [1,2,3]]  
-- = [[(1,4), (1,5), (1,6)], [(2,4), (2,5), (2,6)], [(3,4), (3,5), (3,6)]]  
  
-- "Achatando" com concat (junta as sublistas em uma só)  
resultado = concat [(x,y) | y <- [4,5,6] | x <- [1,2,3]]
```

**Ideia chave:** uma comprehension com 2 geradores é equivalente a uma comprehension externa (no 1º gerador) cujo corpo é outra comprehension (no 2º gerador), tudo concatenado com `concat`.

### 1.3 Guardas (condições/filtros)

```
[x | x <- [1..20], even x]       -- só os pares  
[x | x <- [1..20], x `mod` 3 == 0, x > 5] -- múltiplas condições (AND implícito)
```

#### 1.4 Dependência entre geradores

Geradores posteriores podem usar variáveis dos geradores anteriores:

```
[(x,y) | x <- [1..3], y <- [x..3]]  
-- = [(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)]
```

#### 1.5 replicate com list comprehension (Ex. 2)

```
replicate' :: Int -> a -> [a]  
replicate' n x = [x | _ <- [1..n]]  
  
-- Main> replicate' 3 True = [True, True, True]
```

Usa-se `_` pois não precisamos do valor gerado, só repetir `n` vezes.

#### 1.6 Triplas pitagóricas (Ex. 3)

Condição:  $x^2 + y^2 = z^2$ , com `x,y,z` limite.

```
pyths :: Int -> [(Int, Int, Int)]  
pyths n = [(x,y,z) | x <- [1..n], y <- [1..n], z <- [1..n], x^2 + y^2 == z^2]  
  
-- Main> pyths 10 = [(3,4,5), (4,3,5), (6,8,10), (8,6,10)]
```

#### 1.7 Números perfeitos (Ex. 4)

Um número é perfeito se é igual à soma de seus fatores (divisores próprios, excluindo ele mesmo).

```
fatores :: Int -> [Int]  
fatores n = [x | x <- [1..n-1], n `mod` x == 0]  
  
perfects :: Int -> [Int]  
perfects limite = [x | x <- [1..limite], sum (fatores x) == x]  
  
-- Main> perfects 500 = [6,28,496]
```

#### 1.8 Outros exercícios resolvidos com list comprehension

Produto escalar (Ex. 7):

```
scalarproduct :: [Int] -> [Int] -> Int  
scalarproduct xs ys = sum [x*y | (x,y) <- zip xs ys]  
  
-- Main> scalarproduct [1,2,3] [4,5,6] = 32 (1*4+2*5+3*6)
```

Cubos dos pares (Ex. 12):

```
evenCubes :: Int -> [Int]  
evenCubes limite = [x^3 | x <- [1..limite], even x]
```

```
-- Main> evenCubes 10 = [8,64,216,512]
```

**Inserção ordenada (Ex. 13):**

```
insertOrd :: Int -> [Int] -> [Int]
insertOrd x xs = [y | y <- xs, y < x] ++ [x] ++ [y | y <- xs, y >= x]
```

```
-- Main> insertOrd 4 [0,1,2,5,6] = [0,1,2,4,5,6]
```

**Múltiplos de um número num intervalo (Ex. 14):**

```
howManyMultiples :: Int -> Int -> Int -> Int
howManyMultiples k ini fim = length [x | x <- [ini..fim], x `mod` k == 0]
```

```
-- Main> howManyMultiples 4 1 10 = 2
```

**Separar pares e ímpares (Ex. 18):**

```
separa :: [Int] -> ([Int],[Int])
separa xs = ([x | x <- xs, odd x], [x | x <- xs, even x])
```

```
-- Main> separa [1,4,3,4,6,7,9,10] = ([1,3,7,9],[4,4,6,10])
```

**Converter posições em letras do alfabeto (Ex. 19):**

```
converte :: [Int] -> String
converte xs = ['A'..'Z'] !! (n-1) | n <- xs]
```

```
-- Main> converte [1,2,6,1,9] = "ABFAI"
```

```
-- Main> converte [] = ""
```

**Contar ocorrências de um caractere (Ex. 20):**

```
conta :: String -> Char -> Int
conta xs a = length [c | c <- xs, c == a]
```

```
-- Main> conta "ABCAABCDCA" 'B' = 2
```

**Proliferar inteiros (Ex. 20a):**

```
proliferaInt :: [Int] -> [Int]
proliferaInt xs = concat [replicate n n | n <- xs]
```

```
-- Main> proliferaInt [3,0,2,4,0,1] = [3,3,3,2,2,4,4,4,4,1]
```

**Proliferar caracteres pela posição no alfabeto (Ex. 21):**

```
import Data.Char (ord)
```

```
proliferaChar :: [Char] -> String
proliferaChar xs = concat [replicate (ord c - ord 'A' + 1) c | c <- xs]
```

```
-- Main> proliferaChar "CBD" = "CCCBDDDD"
```

---

## 2. Recursão “manual” (sem list comprehension nem alta ordem)

Padrão geral: caso base (lista vazia) + caso recursivo ( $x:xs$ ).

### 2.1 find usado em positions (Ex. 6)

```
positions :: Eq a => a -> [a] -> [Int]
positions x xs = find x (zip xs [0..n])
  where n = (length xs) - 1
```

```
find :: Eq a => a -> [(a,Int)] -> [Int]
find x ps = [i | (x', i) <- ps, x == x']
```

(Pode ser feito com list comprehension OU recursão pura; o exercício pede para “definir” — qualquer uma das duas é aceitável.)

### 2.2 Exponenciação recursiva (Ex. 8)

Mesmo padrão de recursão da multiplicação (que soma repetidamente):

```
-- multiplicação via soma repetida (referência):
```

```
mult :: Int -> Int -> Int
mult m 0 = 0
mult m n = m + mult m (n-1)
```

```
-- exponenciação via multiplicação repetida:
```

```
(&!) :: Int -> Int -> Int
m &! 0 = 1
m &! n = m * (m &! (n-1))
```

Cálculo de  $2 \ \&! \ 3$ :

```
2 &! 3 = 2 * (2 &! 2)
      = 2 * (2 * (2 &! 1))
      = 2 * (2 * (2 * (2 &! 0)))
      = 2 * (2 * (2 * 1))
      = 8
```

### 2.3 Conversão lista de dígitos → inteiro (Ex. 10)

```
dec2int :: [Int] -> Int
dec2int = foldl (\acc d -> acc * 10 + d) 0
```

```
-- Main> dec2int [2,3,4,5] = 2345
```

(Também pode ser resolvida sem fold: `dec2int xs = sum (zipWith (*) (reverse xs) (map (10^) [0..]))`, mas a versão com `foldl` é a mais elegante e cobre o conteúdo de fold também.)

## 2.4 unfold (Ex. 11)

```
unfold :: (a -> Bool) -> (a -> b) -> (a -> a) -> a -> [b]
unfold p h t x
  | p x      = []
  | otherwise = h x : unfold p h t (t x)
```

Potências de 2 até 2<sup>10</sup>:

```
potenciasDe2 :: [Int]
potenciasDe2 = unfold (> 1024) id (*2) 1
-- = [1,2,4,8,16,32,64,128,256,512,1024]
```

`unfold` é o “dual” do `fold`: em vez de reduzir uma lista a um valor, ela **constrói** uma lista a partir de um valor semente (`x`), parando quando o predicado `p` é satisfeito.

## 2.5 Recursão de cauda/ordem de avaliação (Ex. 22 e 23 — teoria)

```
procuraElemento :: Int -> [Int] -> Bool
procuraElemento n (x:xs) = n == x || procuraElemento n xs
```

```
procuraElemento2 :: Int -> [Int] -> Bool
procuraElemento2 n (x:xs) = procuraElemento2 n xs || n == x
```

- (a) Ambas estão corretas (mesmo resultado), mas a ordem de avaliação difere:
  - `procuraElemento`: testa `n == x` **antes** de recursar — por causa do *short-circuit* do `||`, se encontrar o elemento, **para imediatamente** (não percorre o resto da lista).
  - `procuraElemento2`: chama a recursão **antes** de comparar — ou seja, percorre **toda a lista até o fim** primeiro, e só depois (ao “desempilhar” as chamadas) é que as comparações acontecem, da direita pra esquerda.
- (b) `procuraElemento` é mais eficiente: no melhor caso ela é  $O(1)$  (elemento no início) e não percorre elementos desnecessários; `procuraElemento2` é sempre  $O(n)$ , pois precisa atingir o fim da lista (e construir a pilha de chamadas) antes de comparar qualquer coisa.

```
checkEqual :: Eq a => a -> [a] -> Bool
checkEqual _ [] = True
checkEqual y (z:zs) = (y == z) && checkEqual y zs
```

```

allEqual :: Eq a => [a] -> Bool
allEqual [] = True
allEqual (x:xs) = checkEqual x xs

```

- (a) Ambas usam == para comparar elementos do tipo a; como a é um tipo genérico (polimórfico), o compilador não sabe a priori se esse tipo suporta igualdade — por isso é preciso a restrição de classe Eq a => para garantir que == está definido para a.
- (b) Sem a restrição Eq a, o código não compilaria, pois o Haskell exige que toda operação usada (==) tenha evidência de que o tipo a oferece tal operação. O compilador rejeitaria com erro de tipo (“No instance for Eq a”).

## 2.6 Classe Ord (Ex. 24 — teoria)

- Ord é a classe que define operadores de **ordenação**: <, <=, >, >=, compare, max, min.
- Ela estende Eq (para ser Ord, o tipo já precisa ser Eq).
- É essencial para funções que precisam comparar elementos, não só verificar igualdade — por exemplo: ordenar listas (sort), inserir em estruturas ordenadas (como insertOrd ou árvores binárias de busca), encontrar máximo/mínimo (maximum, minimum).
- Assinatura típica de uma função que a usa: insere :: Ord a => Arvore a -> a -> Arvore a — o Ord a garante que dá para comparar o novo elemento com os já presentes na estrutura, decidindo se vai para a esquerda ou direita.

## 3. Funções de Alta Ordem (Aula 05)

**Definição:** funções que recebem outra função como argumento e/ou retornam uma função como resultado.

### 3.1 map

```

map :: (a -> b) -> [a] -> [b]
map f []      = []
map f (x:xs) = f x : map f xs

```

Aplica f a cada elemento.

```

map (+1) [1,2,3]      = [2,3,4]
map even [1,2,3]      = [False,True,False]
map reverse ["ola","amor"] = ["alo","roma"]

```

Exemplo sem/com map:

```
double :: [Int] -> [Int]
double [] = []
double (x:xs) = (2*x) : double xs

double' xs = map (*2) xs -- equivalente
```

### 3.2 filter

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs)
  | p x = x : filter p xs
  | otherwise = filter p xs
```

Mantém só elementos que satisfazem o predicado.

```
filter even [1,2,3,4,5] = [2,4]
filter (>3) [1,5,2,8] = [5,8]
```

### 3.3 fold (genérico/fold de Hutton) e foldr/foldl (Prelude)

```
fold :: (t -> u -> u) -> u -> [t] -> u
fold f s [] = s
fold f s (a:x) = f a (fold f s x)

fold (+) 0 [1,2,3] = 1 + (2 + (3 + 0)) = 6
```

**foldr** (direita): associa pela direita, acumulador é o 2º argumento de f.

```
foldr :: (t -> u -> u) -> u -> [t] -> u
foldr f s [] = s
foldr f s (x:xs) = f x (foldr f s xs)
```

```
foldr (+) 0 [1,2,3]
= 1 + (2 + (3 + 0)) = 6
```

**foldl** (esquerda): associa pela esquerda, acumulador é o 1º argumento de f.

```
foldl :: (t -> u -> u) -> u -> [t] -> u
foldl f s [] = s
foldl f s (a:x) = foldl f (f s a) x
```

```
foldl (+) 0 [1,2,3]
= foldl (+) (0+1) [2,3]
= foldl (+) (1+2) [3]
= foldl (+) (3+3) []
= 6
```

**Diferença chave (cai bastante em prova):** foldr processa “da direita pra esquerda” (constrói a expressão como f x1 (f x2 (f

`x3 s)))`, `foldl` processa “da esquerda pra direita” mas associando à esquerda (`f (f (f s x1) x2) x3`). Para operações associativas como `+` e `*` dá o mesmo resultado; para outras (como `-`), `(:)`, concatenação de strings em ordem) os resultados podem ser bem diferentes.

Funções implementadas com `fold`:

```
sum'      xs = foldr (+) 0 xs
product' xs = foldr (*) 1 xs
```

Reverter lista com `foldl` (Exercício de Aula 05, item 5):

```
reverse' :: [a] -> [a]
reverse' = foldl (\acc x -> x : acc) []

-- reverse' [1,2,3] = foldl (\acc x -> x:acc) [] [1,2,3]
-- passo 1: acc=[], x=1 -> [1]
-- passo 2: acc=[1] , x=2 -> [2,1]
-- passo 3: acc=[2,1] , x=3 -> [3,2,1]
```

Concat com `foldr` (item 4):

```
concat' :: [[a]] -> [a]
concat' = foldr (++) []

-- concat' [[1,2],[3],[],[4,5]] = [1,2] ++ ([3] ++ ([] ++ ([4,5] ++ []))) = [1,2,3,4,5]
```

### 3.4 Reescrevendo comprehension com `map/filter` (Ex. 9 da Lista 2)

Padrão geral:

```
[f x | x <- xs, p x]      map f (filter p xs)
```

Exemplo do enunciado:

```
[(+7) x | x <- [1..10], odd x]
  map (+7) (filter odd [1..10])
-- [1..10] ímpares: [1,3,5,7,9] -> map (+7): [8,10,12,16,16]...
-- (conferindo: 1+7=8, 3+7=10, 5+7=12, 7+7=14, 9+7=16 => [8,10,12,14,16])
```

Exercícios próprios da Aula 05:

```
quadrados :: [Int] -> [Int]
quadrados = map (^2)

multiplosDe5 :: [Int] -> [Int]
multiplosDe5 = filter (\x -> x `mod` 5 == 0)

quadradoDosPares :: [Int] -> [Int]
quadradoDosPares xs = map (^2) (filter even xs)
```



### 3.5 Exemplo: duplicate e pushRight com alta ordem (Ex. 15 e 16)

```
duplicate :: String -> Int -> String
duplicate s n = concat [s | _ <- [1..n]]
-- ou: duplicate s n = concat (replicate n s)

-- Main> duplicate "ab" 3 = "ababab"
-- Main> duplicate "ab" 0 = ""

pushRight :: String -> Int -> String
pushRight s n = concat [ ">" | _ <- [1 .. (n - length s)] ] ++ s

-- Main> pushRight "abc" 5 = ">>abc"
-- (se n <= length s, a list comprehension gera lista vazia, retornando a própria string s)
```

### 3.6 Inverter lista com alta ordem (Ex. 17)

```
inverte :: [Int] -> [Int]
inverte = foldl (\acc x -> x : acc) []
-- (mesma ideia de reverse' acima)

-- Main> inverte [1,2,3,4,5,6,150] = [150,6,5,4,3,2,1]
```

---

## 4. Cálculo Lambda e Funções Anônimas (Aula 06)

### 4.1 Sintaxe

`\parametro -> expressao`

Uma expressão lambda é um **valor** (assim como 5 ou True); aqui o valor é uma função.

```
somaUm :: Int -> Int
somaUm x = x + 1
```

```
-- equivalente:
\x -> x + 1
```

```
ghci> (\x -> x + 1) 5
6
```

### 4.2 Uso com funções de alta ordem

O principal uso prático: passar funções “descartáveis” direto como argumento de map/filter, sem precisar nomeá-las.

```
-- sem lambda
f x = 2*x + 1
```

```
map f [1,2,3]

-- com lambda (mais conciso, sem poluir namespace)
map (\x -> 2*x + 1) [1,2,3] = [3,5,7]

filter (\s -> length s > 5) ["ola","haskell","top"] = ["haskell"]
```

### 4.3 Múltiplos argumentos

```
\x y -> x + y

ghci> (\x y -> x + y) 3 4
7
```

### 4.4 Casamento de padrão em lambdas

Pode-se usar padrões (tuplas, listas) direto nos parâmetros do lambda:

```
map (\(x,y) -> x + y) [(1,2),(3,4)] = [3,7]
map (\(x:xs) -> x) [[1,2],[3,4]] = [1,3]
```

### 4.5 Exercícios de Aula 06

```
-- 1) tamanhos das strings
map (\s -> length s) ["a","b","c"] -- = [1,2,3]
-- (equivalente a map length [...])

-- 2) múltiplos de 3 e 5 ao mesmo tempo em [1..50]
filter (\x -> x `mod` 3 == 0 && x `mod` 5 == 0) [1..50]
-- = [15,30,45]

-- 3) reter pares (String,Int) cuja string tem mais de 5 caracteres
filtraPares :: [(String,Int)] -> [(String,Int)]
filtraPares = filter (\(s,_) -> length s > 5)
```

## 5. Conectando os conceitos (visão geral / “quando usar o quê”)

| Técnica                   | Quando usar                                                             | Característica                                             |
|---------------------------|-------------------------------------------------------------------------|------------------------------------------------------------|
| <b>List comprehension</b> | Construção declarativa de listas com filtros/transformações simultâneas | Mais legível para múltiplos geradores e guardas combinadas |

| Técnica                                                  | Quando usar                                                                                                           | Característica                                                      |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>map</b>                                               | Transformar cada elemento de uma lista                                                                                | $(a \rightarrow b) \rightarrow [a] \rightarrow [b]$                 |
| <b>filter</b>                                            | Selecionar elementos que satisfazem um predicado                                                                      | $(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$       |
| <b>foldr/foldl</b>                                       | Reduzir/combina uma lista inteira a um único valor (ou outra estrutura)                                               | Difere na associatividade/ordem                                     |
| <b>unfold</b>                                            | “Gerar” uma lista a partir de uma semente (operação inversa do fold)                                                  | Usa predicado de parada + função geradora + função “próximo estado” |
| <b>Lambda (<math>\lambda x \rightarrow \dots</math>)</b> | Função anônima de uso único, normalmente como argumento de <b>map/filter/fold</b>                                     | Evita nomear funções triviais                                       |
| <b>Recursão explícita (<math>x:xs</math>)</b>            | Quando a lógica não se encaixa bem em comprehension/alta-ordem, ou para entender a “mecânica” por trás dessas funções | Caso base + caso recursivo                                          |

Regra de equivalência importante para a prova:

$[f\ x \mid x \leftarrow xs, p\ x]$       `map f (filter p xs)`

## 6. Dica de revisão rápida (checklist antes da prova)

- ☐ Sei escrever list comprehension com **um** e com **múltiplos geradores**.
- ☐ Sei combinar **guardas** (condições) numa comprehension.
- ☐ Sei resolver **pyths**, **perfects**, **replicate** via comprehension de memória.
- ☐ Sei a diferença entre **foldr** e **foldl** e simular a execução passo a passo.
- ☐ Sei implementar **reverse**, **concat**, **sum**, **product** usando fold.
- ☐ Sei escrever e ler uma expressão **lambda**, inclusive com múltiplos parâmetros e padrões (tuplas).
- ☐ Sei transformar  $[f\ x \mid x \leftarrow xs, p\ x]$  em `map f (filter p xs)` e vice-versa.
- ☐ Entendo por que funções polimórficas que usam `==` precisam de `Eq a =>`, e por que comparações de ordem precisam de `Ord a =>`.

- ☐ Sei comparar eficiência/ordem de avaliação entre duas implementações recursivas similares (curto-circuito do `||`/`&&`).
- ☐ Sei usar `unfold` para gerar sequências (ex: potências de 2).

Bons estudos!